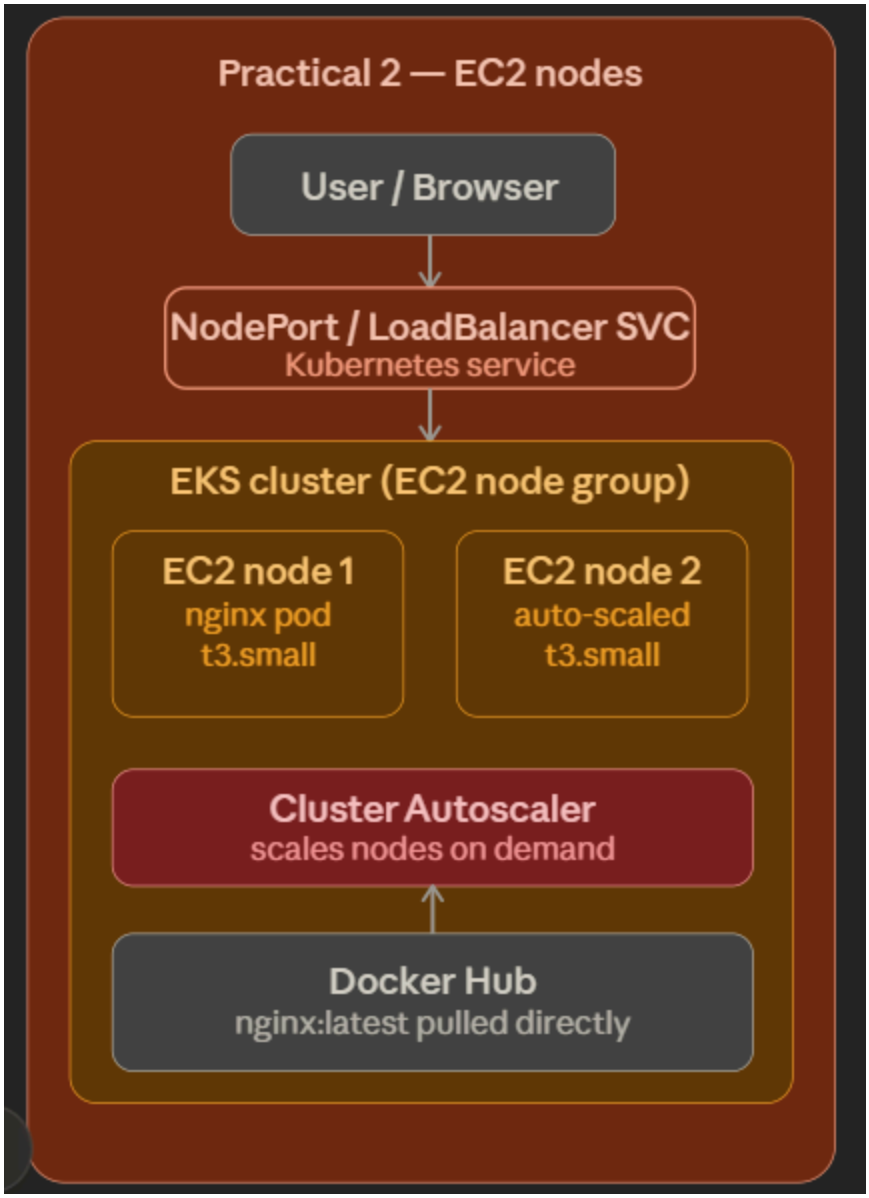
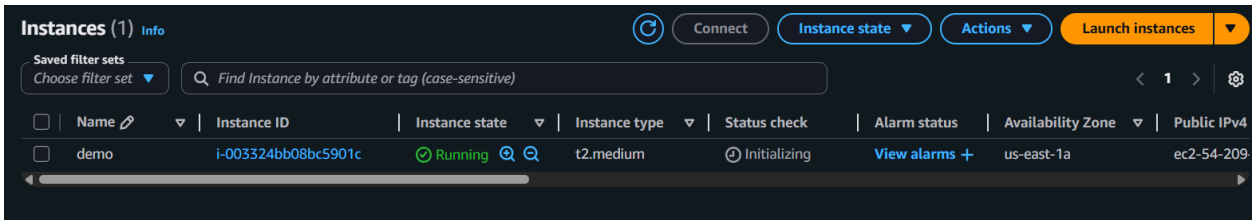


Create 2 role one for ec2 >> administrator
One for EKS role >> it will attach policy automatically



Now lets install everything

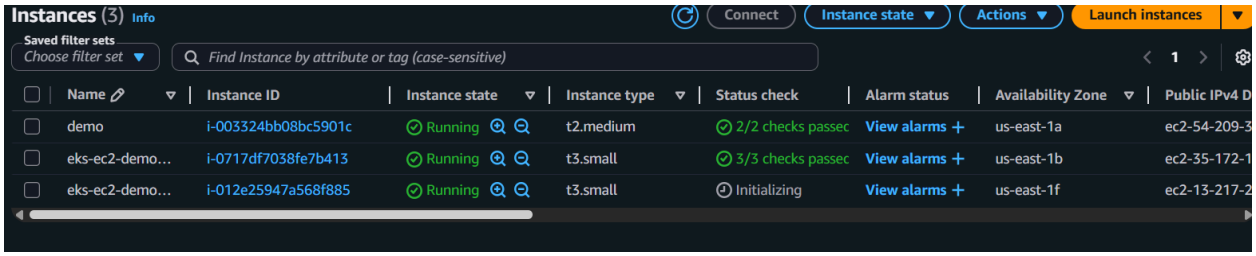
1. Sudo apt update
2. Sudo apt install docker.io
3. Usermod -aG docker ubuntu

```
docker --version
  2  docker ps
  3  aws --version
  4  curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
  5  sudo apt-get install -y unzip
  6  unzip awscliv2.zip
  7  sudo ./aws/install
  8  aws --version
  9  kubectl version --client
 10  curl -LO "https://dl.k8s.io/release/$(curl -sL
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
 11  sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
 12  kubectl version --client
 13  ARCH=amd64
 14  PLATFORM=$(uname -s)_$ARCH
 15  curl -sLO
"https://github.com/eksctl-io/eksctl/releases/latest/download/eksctl_$PLATFORM.tar.gz"
 16  tar -xzf eksctl_$PLATFORM.tar.gz -C /tmp
 17  sudo mv /tmp/eksctl /usr/local/bin
 18  eksctl version
 21  aws sts get-caller-identity
 22 export  ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
 23  export REGION=us-east-1
 24  echo "Account: $ACCOUNT_ID | Region: $REGION"
 25  history
```

Now create a cluster

```
eksctl create cluster \
  --name eks-ec2-demo \
  --region $REGION \
  --version 1.33 \
  --nodegroup-name standard-workers \
  --node-type t3.small \
  --nodes 2 \
  --nodes-min 1 \
  --nodes-max 4 \
  --managed \
  --asg-access
```

It will take 10 to 15 mins to create cluster
Now go to your console and verify everything



Kubectl get nodes

kubectl create namespace prod

kubectl create deployment nginx-app \ --image=nginx:latest \ --replicas=2 \ --namespace=prod

kubectl expose deployment nginx-app \ --type=LoadBalancer \ --port=80 \ --target-port=80 \ --namespace=prod

kubectl get pods -n prod -w

kubectl get svc -n prod # Get the external hostname kubectl get svc nginx-app -n prod \ -o jsonpath='{.status.loadBalancer.ingress[0].hostname}'

```
2026-04-03 14:51:50 [✓]  EKS cluster "eks-ec2-demo" in "us-east-1" region is ready
ubuntu@ip-172-31-29-152:~$ kubectl get nodes
NAME                                       STATUS   ROLES    AGE    VERSION
ip-192-168-25-24.ec2.internal   Ready    <none>   109s   v1.33.8-eks-f69f56f
ip-192-168-35-36.ec2.internal   Ready    <none>   108s   v1.33.8-eks-f69f56f
ubuntu@ip-172-31-29-152:~$ kubectl get nodes -o wide
NAME                                       STATUS   ROLES    AGE    VERSION               INTERNAL-IP     EXTERNAL-IP      OS-IMAGE
KERNEL-VERSION                   CONTAINER-RUNTIME
ip-192-168-25-24.ec2.internal   Ready    <none>   2m2s   v1.33.8-eks-f69f56f   192.168.25.24   13.217.211.156   Amazon Linux 2023.10.20260302
6.12.73-95.123.amzn2023.x86_64   containerd://2.2.1+unknown
ip-192-168-35-36.ec2.internal   Ready    <none>   2m1s   v1.33.8-eks-f69f56f   192.168.35.36   35.172.115.19    Amazon Linux 2023.10.20260302
6.12.73-95.123.amzn2023.x86_64   containerd://2.2.1+unknown
ubuntu@ip-172-31-29-152:~$ kubectl create namespace prod
namespace/prod created
ubuntu@ip-172-31-29-152:~$ kubectl create deployment nginx-app \
  --image=nginx:latest \
  --replicas=2 \
  --namespace=prod
deployment.apps/nginx-app created
ubuntu@ip-172-31-29-152:~$ kubectl expose deployment nginx-app \
  --type=LoadBalancer \
  --port=80 \
  --target-port=80 \
  --namespace=prod
service/nginx-app exposed
ubuntu@ip-172-31-29-152:~$ kubectl get pods -n prod -w
NAME                                READY   STATUS    RESTARTS   AGE
nginx-app-9cb9dd5fd-7cchh   1/1     Running   0          19s
nginx-app-9cb9dd5fd-fzjfd   1/1     Running   0          19s

^Cubuntu@ip-172-31-29-152:~$ kubectl get svc -n prod
NAME        TYPE           CLUSTER-IP      EXTERNAL-IP                                                              PORT(S)        AGE
nginx-app   LoadBalancer   10.100.124.36   a930a1892d4ee4c398db821943bdfeca-672194139.us-east-1.elb.amazonaws.com   80:31513/TCP   27s
ubuntu@ip-172-31-29-152:~$ kubectl get svc nginx-app -n prod \
  -o jsonpath='{.status.loadBalancer.ingress[0].hostname}'
a930a1892d4ee4c398db821943bdfeca-672194139.us-east-1.elb.amazonaws.comubuntu@ip-172-31-29-152:~$
```

Step 7 — Install Cluster Autoscaler

Not secure a930a1892d4ee4c398db821943bdfeca-672194139.us-east-1.elb.amazonaws.com

Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.

Now lets update the images

kubectl set image deployment/nginx-app nginx-app=allenaira/netflix -n prod

Now these will show the name is wrong so

kubectl get all -n prod

kubectl get deployment.apps/nginx-app -o yaml -n prod

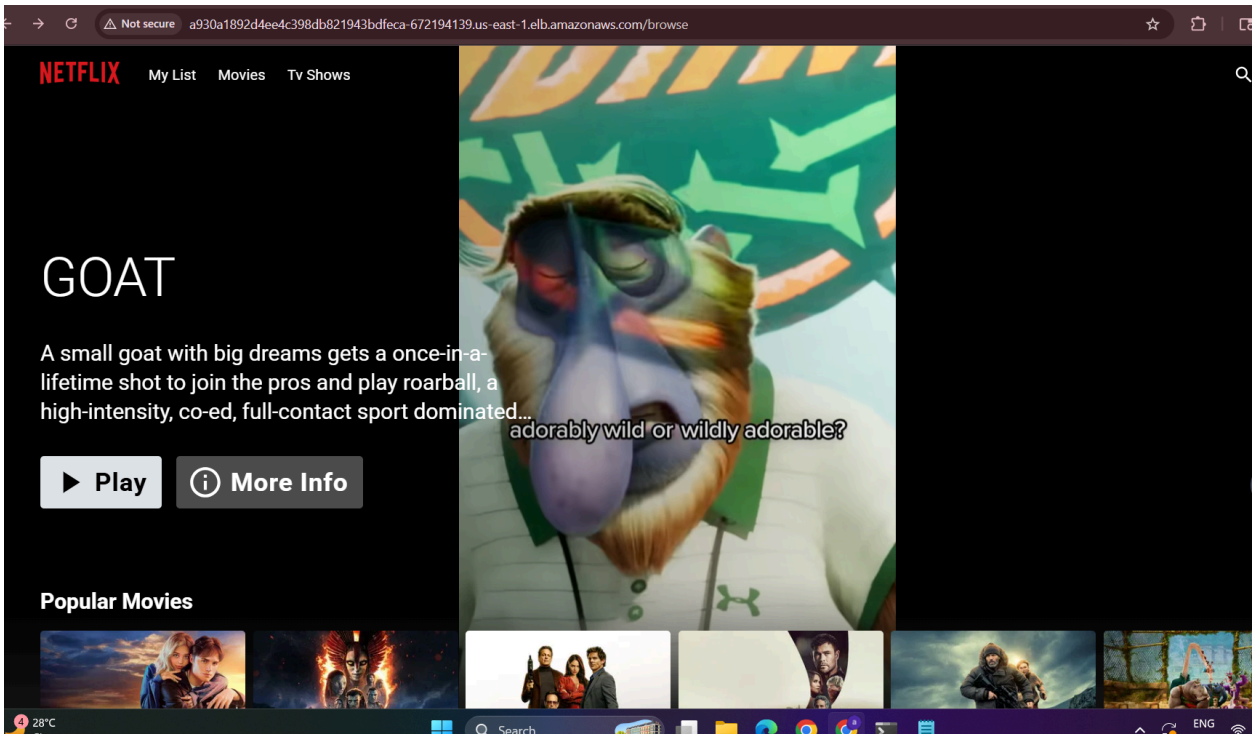
kubectl set image deployment/nginx-app nginx=allenaira/netflix -n prod

kubectl get pods -n prod -w

kubectl rollout restart deployment nginx-app -n prod

kubectl rollout status deployment nginx-app -n prod

Now check on ALB



Delete Kubernetes resources first
kubectl delete namespace prod

Delete the entire cluster (this removes all EC2 nodes, ASG, VPC, etc.)
eksctl delete cluster --name eks-ec2-demo --region $REGION

goto your and confirm if everything is get deleted or not

This is how using the commands you can create the config file
kubectl create deployment nginx-app --image=nginx:latest --replicas=2 --namespace=prod -o yaml >>deploy.yml

And you are done!!!